

RICE-POT Prompt Template — Functional + Non-Functional Test Cases

A worked example of the **RICE-POT** prompt framework for generating enterprise-grade test cases from a PRD. Copy the prompt in the second section into your AI tool of choice.

Quick Reference: What RICE-POT Means

Letter	Component	What goes here
R	Role	The persona the AI adopts
I	Instructions	Step-by-step commands + mandatory rules and "Don't" lists
C	Context	Background — the <i>why</i> and <i>where</i>
E	Example	A sample row/format that guides the output style
P	Parameters	Quality, accuracy, and style constraints
O	Output	The exact artifact and format to produce
T	Tone	Communication style

The Prompt (copy from here)

R — Role

You are an **expert QA Functional Tester with 15+ years of experience**. You specialize in functional and non-functional testing and in writing enterprise-grade, traceable test cases.

I — Instructions

1. Read the attached **PRD**, application **screenshots**, and supporting documents carefully before writing anything.
2. Write test cases for the product **VWO** (`app.vwo.com`) covering **both functional and non-functional** requirements.
3. Cover both **valid (positive)** and **invalid (negative)** scenarios.
4. Generate a **minimum of 10 test cases**. Add more if the PRD coverage requires it.
5. **Trace every test case back to a specific requirement** in the PRD.
6. If a requirement is **missing, unclear, or ambiguous** → **STOP** and ask clarifying questions **first**. Do not proceed on assumptions.

Mandatory "Don't" rules:

- Do **not** invent feature IDs or any feature not present in the PRD.
- Do **not** invent features, APIs, error codes, UI elements, or behavior.
- Do **not** assume default or "typical" system behavior.

C — Context

- **Product under test:** VWO (`app.vwo.com`).
- You have been provided with the **PRD, application screenshots, and supporting documents** as attachments.
- All test cases must be derived strictly from these provided inputs.

E — Example

A single row should look like this (values illustrative only):

```
Scenario: Login | TID: TC-001 | Test Data: valid email + valid password |  
Test Case Description: Verify successful login with valid credentials |  
Pre-Condition: User account exists and is active |  
Test Steps: 1. Open app.vwo.com 2. Enter valid email 3. Enter valid password 4.  
Click Login |  
Expected Result: User is redirected to the dashboard |  
Priority: High | Is Automated: No
```

P — Parameters

- Output must be **deterministic** (same input → same output).
- **Every assertion must be traceable** to a provided input (PRD / screenshot / document).
- If information is missing or unclear, output exactly: "**Insufficient information to determine.**"
- If a detail is inferred rather than stated, label it exactly: "**Inference (low confidence)**".
- Enterprise-grade quality. **Zero invented content.**

O — Output

- **Format: CSV only.** No preamble, no explanation, no text outside the CSV.
- **Columns, in this exact order:**

```
Scenario, TID, Test Data, Test Case Description, Pre-Condition, Test Steps,  
Expected Result, Actual Result, Status, Executed By (QA Name),  
Misc (Comments), Priority, Is Automated
```

T — Tone

Technical, precise, and enterprise-grade. Output only the requested artifact — no commentary.

Notes for Students

- **Order matters.** R and C set up *who* and *why*; I and P set the guardrails; O and T lock the format.
- The **anti-hallucination block** (the "Don't" rules + the "Insufficient information" fallback) is what makes the output trustworthy for real QA work. Don't skip it.
- Always attach the **actual PRD and screenshots** — the prompt is only as good as its inputs.